

1. Что такое Maven и для чего он используется?	3
2. Какое основное преимущество Gradle по сравнению с Maven?	3
3. Чем отличается Maven от Ant?.....	3
1. Как в Maven указать версию Java, используемую в проекте?.....	5
2. Как в Gradle добавить зависимость из центрального репозитория Maven? 5	
3. Как создать мульти-модульный проект в Maven?	5
1. Как настроить кэширование зависимостей в Gradle для ускорения сборки проекта?	8
2. Как использовать профили в Maven для различных сред разработки?	8
3. Как в Gradle выполнить задачу только если условие истинно, используя dynamic task configuration?	8
1. Какова основная цель использования Maven и Gradle в проектах Java?..	12
2. Какие файлы конфигурации используются Maven и Gradle?	12
3. Какое преимущество использования многомодульных проектов?	12
1. Как в Maven добавить зависимость от другого модуля в многомодульном проекте?	14
2. Как в Gradle задать зависимость между модулями в многомодульном проекте?	14
3. Какие есть способы управления версиями зависимостей в многомодульных проектах на Maven и Gradle?	14
1. Как в Gradle настроить многомодульный проект для использования различных версий одной зависимости в разных модулях?	18
2. В чем состоит отличие принципа работы Incremental Build между Maven и Gradle и как это влияет на производительность сборки?	18
3. Как настроить многомодульный проект в Maven для автоматического запуска unit-тестов в зависимых модулях при изменении кода?	18
1. Что такое системы сборки в контексте разработки программного обеспечения?	22
2. Перечислите основные отличия между Maven и Gradle.	22
3. Какова основная функция файла pom.xml в Maven?	22

1. Какие преимущества предоставляет использование Gradle по сравнению с Maven с точки зрения производительности?	24
2. Как в Maven добавить зависимость от сторонней библиотеки?.....	24
3. Опишите, как Gradle использует Groovy (или Kotlin) для конфигурации сборки.	24
1. Как можно настроить multi-module project в Gradle?	27
2. В чем заключается принцип Convention over Configuration, и как он применяется в Maven?	27
3. Как в Gradle реализовать кастомную задачу с использованием Groovy или Kotlin?	27

1. Что такое Maven и для чего он используется?
 2. Какое основное преимущество Gradle по сравнению с Maven?
 3. Чем отличается Maven от Ant?
-

1 Что такое Maven и для чего он используется?

Maven — это инструмент сборки и управления зависимостями Java-проектов.

Он используется для:

- автоматической сборки проекта (compile, test, package)
- управления зависимостями через репозитории
- стандартизации структуры проекта
- управления жизненным циклом сборки
- работы с плагинами (тесты, анализ кода, упаковка)

Ключевая идея: **Convention over Configuration**

→ меньше ручной настройки, больше стандартов.

2 Какое основное преимущество Gradle по сравнению с Maven?

Главное преимущество Gradle — скорость и гибкость.

Почему:

- инкрементальная сборка (пересобирает только изменённое)
- кэширование задач и зависимостей
- DSL на Groovy/Kotlin (логика, условия, циклы)
- лучше масштабируется для больших проектов

Коротко для собеса:

Gradle быстрее и гибче, чем Maven.

3 Чем отличается Maven от Ant?

Критерий	Ant	Maven
Подход	Процедурный (как делать)	Декларативный (что делать)
Управление зависимостями	✗ Нет	✓ Есть
Стандарты структуры	✗ Нет	✓ Есть
Жизненный цикл	✗ Нет	✓ Есть
Конфигурация	Много XML	Меньше XML

Ant:

- просто выполняет команды
- всё нужно описывать вручную
- нет стандартов и зависимостей

Maven:

- управляет зависимостями
- имеет фиксированный lifecycle
- навязывает структуру проекта

Коротко:

Ant — “делай сам”, **Maven** — “по стандарту”.

1. Как в Maven указать версию Java, используемую в проекте?
 2. Как в Gradle добавить зависимость из центрального репозитория Maven?
 3. Как создать мульти-модульный проект в Maven?
-

1 Как в Maven указать версию Java, используемую в проекте?

Самый правильный и ожидаемый способ — через **maven-compiler-plugin**.

Вариант 1 (классический, универсальный)

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>
```

Maven сам подставит эти значения в **maven-compiler-plugin**.

Вариант 2 (явно через плагин)

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Вариант 3 (современный, рекомендованный)

```
<properties>
  <maven.compiler.release>17</maven.compiler.release>
</properties>
```



Почему лучше **release**?

Он гарантирует совместимость API (а не только байткода).

2 Как в Gradle добавить зависимость из центрального репозитория Maven?

1. Подключаем Maven Central

```
repositories {  
    mavenCentral()  
}
```

2. Добавляем зависимость

```
dependencies {  
    implementation 'org.springframework:spring-context:6.1.2'  
}
```

Kotlin DSL (build.gradle.kts)

```
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    implementation("org.springframework:spring-context:6.1.2")  
}
```

✦ По умолчанию Gradle уже умеет работать с Maven-репозиториями.

3 Как создать мульти-модульный проект в Maven?

1 Родительский проект (parent)

packaging = pom

```
<project>  
    <modelVersion>4.0.0</modelVersion>  
  
    <groupId>com.example</groupId>  
    <artifactId>parent</artifactId>  
    <version>1.0.0</version>  
    <packaging>pom</packaging>  
  
    <modules>  
        <module>service</module>  
        <module>api</module>  
    </modules>  
</project>
```

2 Модуль (например `service`)

```
<project>
  <parent>
    <groupId>com.example</groupId>
    <artifactId>parent</artifactId>
    <version>1.0.0</version>
  </parent>

  <artifactId>service</artifactId>
</project>
```

3 Структура проекта

```
parent/
├── pom.xml
├── service/
│   └── pom.xml
└── api/
    └── pom.xml
```

Зачем нужен parent?

- единые версии зависимостей
 - общие плагины
 - единая версия Java
 - централизованное управление конфигурацией
-

Коротко для собеседования

- **Java версия в Maven** → `maven.compiler.release`
- **Зависимость в Gradle** → `repositories + dependencies`
- **Мульти-модуль Maven** → `packaging=pom + modules`

1. Как настроить кэширование зависимостей в Gradle для ускорения сборки проекта?
 2. Как использовать профили в Maven для различных сред разработки?
 3. Как в Gradle выполнить задачу только если условие истинно, используя dynamic task configuration?
-

1 Как настроить кэширование зависимостей в Gradle для ускорения сборки проекта?

◆ 1. Кэш зависимостей (по умолчанию есть)

Gradle уже кэширует зависимости в:

```
~/.gradle/caches
```

Но поведение можно улучшить.

◆ 2. Управление временем жизни зависимостей

По умолчанию:

- release → кэшируются навсегда
- SNAPSHOT → проверяются каждые 24 часа

Можно изменить:

```
configurations.all {
    resolutionStrategy {
        cacheDynamicVersionsFor 10, 'minutes'
        cacheChangingModulesFor 10, 'minutes'
    }
}
```

✦ Полезно для CI и SNAPSHOT-зависимостей.

◆ 3. Включение Build Cache (ключевое ускорение)

```
# gradle.properties
org.gradle.caching=true
```

- ✓ Кэширует **результаты задач**, а не только зависимости
- ✓ Повторные сборки сильно ускоряются

◆ 4. Параллельная сборка

```
org.gradle.parallel=true
```

◆ 5. Configuration Cache (очень важно)

```
org.gradle.configuration-cache=true
```

- ✓ Кэширует фазу конфигурации
 - ✓ Даёт огромный прирост на больших проектах
-

✚ Итог для собеседования

Gradle ускоряется за счёт **dependency cache + build cache + configuration cache**.

2 Как использовать профили в Maven для различных сред разработки?

Профили **Maven** позволяют менять конфигурацию под:

- dev / test / prod
 - локальную сборку / CI
-

◆ Пример профиля

```
<profiles>
  <profile>
    <id>dev</id>
    <properties>
      <spring.profiles.active>dev</spring.profiles.active>
    </properties>
  </profile>

  <profile>
    <id>prod</id>
    <properties>
      <spring.profiles.active>prod</spring.profiles.active>
    </properties>
  </profile>
</profiles>
```

◆ Активация профиля

Через команду:

```
mvn clean package -Pdev
```

По умолчанию:

```
<profile>
  <id>dev</id>
  <activation>
    <activeByDefault>true</activeByDefault>
  </activation>
</profile>
```

◆ Активация по условию

```
<activation>
  <property>
    <name>env</name>
    <value>prod</value>
  </property>
</activation>
mvn clean package -Denv=prod
```

📌 Для собеса

Профили Maven — это способ менять зависимости, плагины и свойства под разные окружения.

3 Как в Gradle выполнить задачу только если условие истинно (dynamic task configuration)?

◆ `onlyIf` — самый частый и правильный вариант

```
tasks.register('deploy') {
  onlyIf {
    project.hasProperty("deploy")
  }

  doLast {
    println "Deploying..."
  }
}
```

Запуск:

```
gradle deploy -Pdeploy
```

◆ Условная регистрация задачи (на этапе конфигурации)

```
if (project.hasProperty("ci")) {  
    tasks.register('ciBuild') {  
        doLast {  
            println "CI build"  
        }  
    }  
}
```

◆ Условие через environment variable

```
tasks.register('prodTask') {  
    onlyIf {  
        System.getenv("ENV") == "prod"  
    }  
}
```

📌 Важное отличие (любят спрашивать)

Подход **Когда выполняется**

`if` во время конфигурации

`onlyIf` во время **execution**

Правильный ответ:

Для **dynamic task configuration** лучше использовать `onlyIf`, а не `if`.

🔥 Итог коротко (идеально для ответа)

- **Gradle caching** → dependency cache + build cache + configuration cache
- **Maven profiles** → разные конфиги для dev/test/prod
- **Gradle condition** → `onlyIf { ... }`

1. Какова основная цель использования Maven и Gradle в проектах Java?
 2. Какие файлы конфигурации используются Maven и Gradle?
 3. Какое преимущество использования многомодульных проектов?
-

1 Какова основная цель использования Maven и Gradle в Java-проектах?

Основная цель — автоматизация сборки и управления зависимостями.

Оба инструмента позволяют:

- управлять зависимостями и их версиями
- собирать проект (compile → test → package)
- стандартизировать процесс сборки
- подключать плагины (тесты, анализ, упаковка, деплой)
- упростить CI/CD

Коротко для собеседа:

Maven и Gradle нужны для повторяемой, автоматизированной и стандартизированной сборки Java-проектов.

2 Какие файлы конфигурации используются Maven и Gradle?

◆ Maven

- `pom.xml` — основной файл конфигурации
Содержит:
 - зависимости
 - плагины
 - версии
 - профили
 - модули

✚ XML, декларативный подход.

◆ Gradle

- `build.gradle` или `build.gradle.kts` — основной файл сборки
- `settings.gradle` или `settings.gradle.kts` — настройки проекта и модулей
- `gradle.properties` — общие параметры сборки

✚ DSL на Groovy или Kotlin, более гибкий и программируемый.

Коротко

Инструмент	Основной файл
Maven	<code>pom.xml</code>
Gradle	<code>build.gradle(.kts)</code>

3 Какое преимущество использования многомодульных проектов?

Главное преимущество — структурирование и переиспользование кода.

Многомодульные проекты позволяют:

- разделить проект на логические части (`api`, `service`, `core`)
 - переиспользовать код между модулями
 - централизованно управлять версиями зависимостей
 - собирать весь проект одной командой
 - ускорять сборку (особенно в Gradle)
-

Типичный пример

```
project-root
├── api
├── service
├── persistence
└── common
```

Для собеседования

Многомодульность улучшает масштабируемость, поддержку и переиспользование кода, снижая дублирование и упрощая управление зависимостями.

1. Как в Maven добавить зависимость от другого модуля в многомодульном проекте?
 2. Как в Gradle задать зависимость между модулями в многомодульном проекте?
 3. Какие есть способы управления версиями зависимостей в многомодульных проектах на Maven и Gradle?
-

1 Как в Maven добавить зависимость от другого модуля в многомодульном проекте?

В Maven модуль подключается как обычная зависимость.

Пример

Есть модули:

- common
- service

В service/pom.xml:

```
<dependencies>
  <dependency>
    <groupId>com.example</groupId>
    <artifactId>common</artifactId>
    <version>${project.version}</version>
  </dependency>
</dependencies>
```

 Важно:

- groupId и version берутся из parent pom.xml
- \${project.version} гарантирует синхронность версий

! Типичная ошибка

Добавлять модуль в <modules> — это не зависимость, а только участие в сборке.

2 Как в Gradle задать зависимость между модулями в многомодульном проекте?

В Gradle используется **project dependency**.

Пример структуры

```
root
├── settings.gradle
├── common
└── service
```

settings.gradle

```
include("common", "service")
```

service/build.gradle

```
dependencies {
    implementation project(":common")
}
```

📌 Gradle сам:

- определит порядок сборки
 - соберёт `common` раньше `service`
-

3 Какие есть способы управления версиями зависимостей

в многомодульных проектах Maven и Gradle?

◆ Maven

1 `dependencyManagement` (основной и правильный)

В parent `pom.xml`:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>6.1.2</version>
    </dependency>
  </dependencies>
</dependencyManagement>
```

В модулях:

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
</dependency>
```

- ✓ Версия задаётся один раз
 - ✓ В модулях версии не дублируются
-

2 Properties

```
<properties>
  <spring.version>6.1.2</spring.version>
</properties>
```

3 BOM (Bill of Materials)

```
<dependencyManagement>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-dependencies</artifactId>
    <version>3.2.0</version>
    <type>pom</type>
    <scope>import</scope>
  </dependency>
</dependencyManagement>
```

📌 Часто используют с Spring Boot.

◆ Gradle

1 version catalog (рекомендованный способ)

gradle/libs.versions.toml

```
[versions]
spring = "6.1.2"
```

```
[libraries]
spring-context = { module = "org.springframework:spring-context", version.ref
= "spring" }
```

В build.gradle:

```
dependencies {
  implementation(libs.spring.context)
}
```

2 ext / extra properties

```
ext {  
    springVersion = "6.1.2"  
}
```

3 Dependency constraints

```
dependencies {  
    constraints {  
        implementation("org.springframework:spring-context:6.1.2")  
    }  
}
```

4 BOM

```
dependencies {  
    implementation(platform("org.springframework.boot:spring-boot-  
dependencies:3.2.0"))  
}
```

Коротко для собеседования

- **Maven module dependency** → обычный <dependency>
- **Gradle module dependency** → implementation project(":module")
- **Maven versions** → dependencyManagement / BOM
- **Gradle versions** → version catalog / constraints / BOM

1. Как в Gradle настроить многомодульный проект для использования различных версий одной зависимости в разных модулях?
 2. В чем состоит отличие принципа работы Incremental Build между Maven и Gradle и как это влияет на производительность сборки?
 3. Как настроить многомодульный проект в Maven для автоматического запуска unit-тестов в зависимых модулях при изменении кода?
-

1 Как в Gradle использовать разные версии одной зависимости в разных модулях?

👉 Короткий ответ:

Gradle позволяет разным модулям иметь разные версии одной зависимости, если они не сходятся в одном classpath.

◆ Вариант 1 — просто указать версии в модулях (самый частый)

```
// module-a/build.gradle
dependencies {
    implementation("com.fasterxml.jackson.core:jackson-databind:2.15.2")
}

// module-b/build.gradle
dependencies {
    implementation("com.fasterxml.jackson.core:jackson-databind:2.13.5")
}
```

✓ Работает, если:

- модули не зависят друг от друга напрямую
 - или dependency не «протекает» вверх
-

◆ Вариант 2 — `dependency constraints` на уровне модуля

```
dependencies {
    constraints {
        implementation("com.fasterxml.jackson.core:jackson-databind:2.15.2")
    }
}
```

✦ Удобно, если часть зависимостей общая, а версии разные.

◆ Вариант 3 — Version Catalog + override

```
[libraries]
jackson = { module = "com.fasterxml.jackson.core:jackson-databind", version =
"2.15.2" }
dependencies {
    implementation(libs.jackson)
}
```

В другом модуле:

```
dependencies {
    implementation("com.fasterxml.jackson.core:jackson-databind:2.13.5")
}
```

! Важное ограничение (часто спрашивают)

Если **модуль А** зависит от **В**, и оба используют одну библиотеку:

- Gradle выберет **одну версию** (обычно более новую)
 - разные версии **не могут одновременно существовать** в одном classpath
-

2 Отличие Incremental Build в Maven и Gradle и влияние на производительность

◆ Maven — ограниченный инкрементальный подход

- Ориентирован на **lifecycle + плагины**
- При изменении файла часто пересобирается **вся фаза**
- Плагины в основном **не инкрементальные**
- Нет полноценного кэша результатов задач

✦ Итог:

Maven часто пересобирает больше, чем нужно

◆ Gradle — task-based incremental build

- Каждая задача имеет:
 - входы (inputs)
 - выходы (outputs)
- Если входы не изменились → задача **пропускается**
- Поддержка:
 - incremental build
 - build cache
 - configuration cache

✦ Итог:

Gradle пересобирает **только реально изменённое**

🔥 Коротко для собеседования

Gradle инкрементален на уровне задач, Maven — на уровне фаз. Поэтому Gradle заметно быстрее на больших проектах.

3 Как в Maven настроить автоматический запуск unit-тестов в зависимых модулях при изменении кода?

👉 Короткий ответ:

Maven делает это **автоматически**, если проект корректно многомодульный.

◆ Что происходит по умолчанию

- Модули связаны через `<dependency>`
- При `mvn test` или `mvn install`:
 1. Пересобирается изменённый модуль
 2. Пересобираются **все зависящие от него модули**
 3. В каждом из них запускаются unit-тесты

✓ Дополнительной настройки **не требуется**

◆ Важные условия

- Модули объявлены в `<modules>` родителя
- Зависимости оформлены через `<dependency>`
- Используется стандартный `lifecycle`

◆ Управление тестами (опционально)

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins>
</build>
```

◆ Частый вопрос на собеседе

Как прогнать только нужные модули?

```
mvn -pl service -am test
```

- `-pl` — выбранный модуль
 - `-am` — также собрать зависимости
-

🔥 Итог (идеальный ответ)

- **Gradle:** разные версии зависимостей возможны, пока не сходятся в одном classpath
- **Incremental Build:**
Maven — фазовый, Gradle — task-based
- **Maven tests:**
тесты в зависимых модулях запускаются автоматически при сборке

1. Что такое системы сборки в контексте разработки программного обеспечения?
 2. Перечислите основные отличия между Maven и Gradle.
 3. Какова основная функция файла pom.xml в Maven?
-

1 Что такое системы сборки в контексте разработки ПО?

Система сборки — это инструмент, который автоматизирует процесс превращения исходного кода в готовый артефакт.

Она отвечает за:

- компиляцию исходного кода
- управление зависимостями
- запуск тестов
- упаковку (JAR / WAR / EAR)
- выполнение вспомогательных задач (линтинг, анализ, деплой)
- воспроизводимость сборки (одинаковый результат на любой машине)

Коротко:

Система сборки автоматизирует сборку, тестирование и упаковку проекта.

2 Основные отличия между Maven и Gradle

Критерий	Maven	Gradle
Язык конфигурации	XML	DSL (Groovy / Kotlin)
Подход	Декларативный	Декларативный + программируемый
Управление зависимостями	Есть	Есть
Скорость сборки	Медленнее	Быстрее
Инкрементальная сборка	Ограниченная	Полноценная
Кэширование	Ограничено	Build cache, configuration cache
Гибкость	Ниже	Выше
Многомодульность	Хорошая	Отличная

Коротко для собеса:

Maven проще и стандартизирован, Gradle быстрее и гибче.

3 Какова основная функция файла `pom.xml` в Maven?

`pom.xml` (Project Object Model) — это центральный конфигурационный файл Maven-проекта.

Он:

- описывает проект (groupId, artifactId, version)
- управляет зависимостями
- настраивает плагины
- определяет lifecycle сборки
- содержит профили и модули
- управляет версиями и настройками проекта

Коротко:

`pom.xml` описывает проект и управляет всем процессом сборки в Maven.



Если ответить одной фразой (часто просят):

`pom.xml` — это декларативное описание проекта и его сборки.

1. Какие преимущества предоставляет использование Gradle по сравнению с Maven с точки зрения производительности?
 2. Как в Maven добавить зависимость от сторонней библиотеки?
 3. Опишите, как Gradle использует Groovy (или Kotlin) для конфигурации сборки.
-

1 Какие преимущества Gradle даёт по производительности по сравнению с Maven?

Gradle значительно быстрее Maven на средних и больших проектах.

Основные причины:

◆ Инкрементальная сборка на уровне задач

- Каждая задача имеет **inputs / outputs**
 - Если входы не изменились → задача **пропускается**
 - Maven чаще пересобирает целые фазы lifecycle
-

◆ Build Cache

- Кэшируются **результаты выполнения задач**
- Повторная сборка или сборка на CI может взять результат из кэша

```
org.gradle.caching=true
```

◆ Configuration Cache

- Кэшируется фаза конфигурации
- Особенно ускоряет большие многомодульные проекты

```
org.gradle.configuration-cache=true
```

◆ Параллельная сборка

```
org.gradle.parallel=true
```

◆ Lazy configuration

- Задачи конфигурируются **только если нужны**
- В Maven конфигурация выполняется всегда

Коротко для собеседования

Gradle быстрее за счёт task-based incremental build, кэширования и ленивой конфигурации.

2 Как в Maven добавить зависимость от сторонней библиотеки?

Зависимость добавляется в файл `pom.xml` в секцию `<dependencies>`.

Пример

```
<dependencies>
  <dependency>
    <groupId>org.apache.commons</groupId>
    <artifactId>commons-lang3</artifactId>
    <version>3.14.0</version>
  </dependency>
</dependencies>
```

Maven:

- сам скачивает библиотеку из репозитория (по умолчанию Maven Central)
- положит её в локальный кэш (`~/.m2/repository`)
- добавит в `classpath` проекта

Если репозиторий не Central

```
<repositories>
  <repository>
    <id>custom-repo</id>
    <url>https://repo.example.com/maven2</url>
  </repository>
</repositories>
```

3 Как Gradle использует Groovy / Kotlin для конфигурации сборки?

Gradle использует DSL (Domain-Specific Language) на:

- **Groovy** → `build.gradle`
- **Kotlin** → `build.gradle.kts`

Это настоящие языки программирования, а не просто конфигурация.

◆ Пример (Groovy DSL)

```
plugins {  
    id 'java'  
}  
  
dependencies {  
    implementation 'org.apache.commons:commons-lang3:3.14.0'  
}
```

◆ Пример (Kotlin DSL)

```
plugins {  
    java  
}  
  
dependencies {  
    implementation("org.apache.commons:commons-lang3:3.14.0")  
}
```

◆ Что это даёт?

- условия (if)
- циклы
- функции
- переменные
- переиспользуемые скрипты
- динамическую конфигурацию задач

```
tasks.register("envTask") {  
    doLast {  
        println("ENV = ${System.getenv("ENV")}")  
    }  
}
```

📌 Ключевое отличие от Maven

Maven — декларативный XML, Gradle — программируемая конфигурация.

🔥 Итог одной фразой (идеально для интервью)

- **Gradle быстрее Maven** за счёт инкрементальности и кэшей
- **Maven dependency** добавляется через `<dependency>` в `pom.xml`
- **Gradle конфигурируется через Groovy/Kotlin DSL**, что даёт гибкость и динамику

1. Как можно настроить multi-module project в Gradle?
 2. В чем заключается принцип Convention over Configuration, и как он применяется в Maven?
 3. Как в Gradle реализовать кастомную задачу с использованием Groovy или Kotlin?
-

1 Как настроить multi-module project в Gradle?

◆ 1. Структура проекта

```
root-project
├── settings.gradle
├── build.gradle
├── module-a
│   └── build.gradle
└── module-b
    └── build.gradle
```

◆ 2. settings.gradle — объявляем модули

```
rootProject.name = "demo"
include("module-a", "module-b")
```

◆ 3. Общая конфигурация в root build.gradle

```
allprojects {
    repositories {
        mavenCentral()
    }
}

subprojects {
    apply plugin: "java"

    java {
        toolchain {
            languageVersion = JavaLanguageVersion.of(17)
        }
    }
}
```

◆ 4. Зависимость между модулями

```
// module-b/build.gradle
dependencies {
    implementation project(":module-a")
}
```

✦ Gradle сам определит порядок сборки.

2 В чем принцип Convention over Configuration и как он применяется в Maven?

Convention over Configuration — это принцип, при котором:

инструмент предполагает разумные значения по умолчанию, а разработчик настраивает только отклонения от них.

◆ Как это выглядит в Maven

Maven жёстко задаёт соглашения:

Что	По умолчанию
Исходники	src/main/java
Ресурсы	src/main/resources
Тесты	src/test/java
Цикл сборки	compile → test → package → install
Имя артефакта	\${artifactId}-\${version}.jar

➡ Если структура соблюдена, `pom.xml` может быть **очень минимальным**.

```
<project>
  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0.0</version>
</project>
```

✦ В отличие от Ant, где всё настраивается вручную.

3 Как в Gradle реализовать кастомную задачу (Groovy / Kotlin)?

◆ Вариант 1 — простая задача (Groovy DSL)

```
tasks.register("hello") {
    doLast {
        println("Hello from Gradle")
    }
}
```

◆ Вариант 2 — Kotlin DSL

```
tasks.register("hello") {
    doLast {
        println("Hello from Gradle")
    }
}
```

◆ Вариант 3 — кастомный тип задачи (Groovy)

```
abstract class PrintTask extends DefaultTask {

    @Input
    String message

    @TaskAction
    void print() {
        println(message)
    }
}

tasks.register("printMsg", PrintTask) {
    message = "Custom Gradle task"
}
```

◆ Вариант 4 — Kotlin (типобезопасный)

```
abstract class PrintTask : DefaultTask() {

    @get:Input
    abstract val message: Property<String>

    @TaskAction
    fun print() {
        println(message.get())
    }
}

tasks.register<PrintTask>("printMsg") {
    message.set("Custom Gradle task")
}
```

📌 Что важно сказать на собеседовании

- Gradle-задачи имеют **inputs / outputs**
- Это позволяет **incremental build** и **caching**
- Kotlin DSL даёт лучшую типизацию и автокомплит

🔥 Итог кратко

- **Multi-module Gradle** → `settings.gradle + project(":module")`
- **Convention over Configuration** → минимум конфигурации при соблюдении стандартов (Maven)
- **Custom task Gradle** → `tasks.register + @TaskAction`